Core JavaScript Cheatsheet — Ch06 (P1–64) | Page 1/2

1. JS简介 / 事件驱动 / 浏览器 (P2–9)

- 高级/动态/无类型/解释型；标准=**ECMAScript**；Web三核心HTML+CSS+JS (P2)
- **原型式prototype-based**，函数一等公民first-class → OOP/命令式/函数式 (P2)
- **事件驱动Event-driven**(P4)：用户操作(点击/按键)=event，JS响应事件

⚠ 关键验证必须服务器端！客户端验证可被绕过(复制页面删除代码)(P4)

```
// 三种引入(P7): ①onclick="..." ②<script>...</script> ③<script src="">
// IO(P9): console.log("调试"); alert("弹窗"); prompt("输入?")→字符串/null
// DevTools(P8): Sources浏览代码 | Console交互 | Breakpoints断点
```

2. 变量 & 语法 (P11–13)

命名: $/_/字母开头，区分大小写，camelCase。保留字(P12)不可用

```
let z = x + y;       // 块级Block,可变 ✓默认选择
const PI = 3.14;     // 块级Block,不可变 ✓✓优先使用
var flag = true;     // 函数级Function ✗避免
zz = z;              // 全局Global! ✗✗危险!
let a, b, c=10;     // 同行多变量, a和b=undefined
```

⚠ ASI陷阱(P13): `return`后换行→自动插；→返回undefined! `return↵x; → return; x;`

Q: const arr=[1,2]; arr.push(3); 报错吗？ A: 不报错！const阻止**变量**重赋值(arr=...)，不阻止修改**对象内容**(.push)。

3. 类型 & typeof (P14, P21–22)

原始Primitives: Number, String, Boolean, null, undefined → 栈上直接存值
对象Objects: Function, Array, Date, Math → 栈上存引用,堆上存数据; 无引用自动GC

```
let a=10,b=a; b=20;      // a=10(不变) — 原始:复制值
let o={x:10},p=o; p.x=20; // o.x=20(变了!) — 对象:复制引用
```

null=故意留空(有架子没纸) | **undefined**=未声明/未赋值(连架子都没有)(P21)

`typeof 42`→"number"	`typeof {}`→"object"	`typeof null`→**"object"!**
`typeof "s"`→"string"	`typeof fn`→"function"	`typeof NaN`→**"number"!**
`typeof true`→"boolean"	`typeof []`→"object"	`typeof undef`→"undefined"

4. Number (P15–17)

全为**64位双精度浮点IEEE754**；`0.1+0.2≠0.3`；运算`+ - * / % ++ -- += -=`

```
1/0→Infinity; parseInt("124")→124; parseInt("3.99")→3(截断!)
parseInt("hello")→NaN; Math.sin(3.5); Math.round(4.7)→5
// NaN(P17): 传染性NaN+5=NaN; NaN===NaN→false! 用isNaN()检测
```

5. String (P18–19)

Unicode16位序列；单双引号均可无区别；转义`\' \" \\ \n \t`；**无char类型**

`charAt(n)` 第n字符	`indexOf(s)` 子串位置	`substring(a,b)` 截取
`toLowerCase/toUpperCase`	`.length` 长度	`.toString()` 数→串

```
+"123"→123(串→数); "a"+"b"→"ab"(拼接); "abc".charAt(0)→"a"
```

6. Boolean(P20) & null/undefined(P21)

`true/false`; `!`(NOT) `&&`(AND) `||`(OR); **&&和||短路求值**

```
0 && "hello"→0(falsy直接返回)  1 && "hello"→"hello"(返回第二个)
"" || "default"→"default"     "hi" || "default"→"hi"
```

7. 隐式类型转换 Type Coercion (P23–26) ★★

→Number(P23): true→1 false→0 Str→数值/NaN null→0 undefined→NaN

```
4-true→3  7*"3"→21  4+null→4  4+undefined→NaN
```

→String(P23): Number→"值" Bool→"true"/"false" null→"null"

→Boolean(P24) ★★必考

Falsy(仅5个): `0, "", undefined, NaN, null`

Truthy: 其余一切，**包括"0"和"false"！**（非空字符串）和`[] {}`（对象）

```
if(obj){obj.fn();}          // 安全访问(P24)
let name = other || "default"; // 默认值(P24)
Number/String/Boolean(...)   // 手动转换(P24)
```

== vs === (P25) ★★

```
"dog"=="dog"→T  1==true→T!   'abc'==['abc']→T!!  // ==会转类型
"dog"=="dog"→T 1===true→F   'abc'===['abc']→F   // ===不转,默认用这个!
```

经典陷阱(P26)

```
2-"1"→1(数学)  2+"1"→"21"(拼接!)  "b"+"a"+ +"a"+"a"→"baNaNa"
''==0→T  0=='0'→T  ''=='0'→F  ← ==非传递!A==B∧B==C ⇏ A==C
```

Q: "5"+3=? "5"-3=? true+true=? Boolean("0")=?
A: "53"(拼接) | 2(数学) | 2(true→1,1+1) | **true**("0"非空串!)

8. 控制结构 (P27–28)

```
for(let i=0;i<n;i++){...}          // for(P27)
for(let key in obj){obj[key];}     // for-in遍历属性名key!(P27)
while(cond){...}                   // while(P27)
if(a){..}else if(b){..}else{..}    // if-else(P28)
switch(v){case 1:..break;default:..}// switch ⚠必须break否则穿透!(P28)
let r = cond ? valT : valF;        // 三元运算符(P28)
```

9. Arrays 数组 (P29–34)

```
let a=[1,"two",true]; // 混合类型! 索引从0, .length=3(P30)
a[99]="x"; a.length;  // 100! 自动扩展(P32)
a[50];                // undefined — 不报错!(P32)
Array(10)→空len=10; Array(10,2)→[10,2](P31) // ⚠行为不同!
```

方法(P33) **★必记**

`push/pop` 末尾加/删	`unshift/shift` 头部加/删	`join`→字符串
`reverse` 反转(改原)	`sort(fn?)` 排序	`concat` 连接(不改原)
`slice(a,b)` 截取(不改原)	`splice` 插入(改原!)	`delete`→undefined(len不变)

⚠ sort()默认按**字符串**排! `[10,9,80].sort()→[10,80,9]` 错! 需`.sort((a,b)=>a-b)`

关联数组(P34): 字符串索引=Object, 无push/pop

Core JavaScript Cheatsheet — Ch06 (P1–64) | Page 2/2

10. Functions 函数 (P35–42)

```
function name(p1,p2){return r;} // 须先定义再使用(P36)
```

Call by Value vs Call by Sharing (P37) ★★

类型	传递	改属性/push	重赋值参数
原始Primitive	值副本	✗不影响原值	✗不影响
对象Object	引用副本	✓**影响原对象!**	✗不影响原引用

```
// Value(P37): let u=1,v=run(u); // u=1(不变),v=2
// Sharing(P37课件):
function setOut(p1){
  p1.push("Ivan");   // ✓改了原数组!
  p1 = newArray;     // ✗重赋值无效!
} // p1=["Isaac","Garet","Ivan"]
```

Q: f(arr){arr.push(1);arr=[9];} a=[0]; f(a); a=?
A: `[0,1]` push有效(改原)，赋值无效(只改函数内引用)

参数(P38-39) / 一等公民(P40) / 匿名&箭头(P41) / 递归(P42)

```
// 参数: 无类型;少→undefined;多→arguments获取全部(P38-39)
function findMax(){/*遍历arguments*/}; findMax(4,5,6)→6(P39)
// 一等公民: fn不加()→函数本身; fn()→调用得返回值(P40)
let x = announce;   // x是函数  |  let y = announce(); // y是返回值
// 匿名+箭头(P41): arr.sort((a,b)=>{return b-a;});
// sort: 负→a前,正→b前,0→不变
// 递归(P42): function cry(n){return n>0?cry(n-1)+"a":"hiy"}
// ninja.yell(5)→"hiyaaaaa"
```

11. Objects 对象 (P43–49)

对象=名值对(名String,值任意) ≈ HashMap/Dict(P44); **数组函数也是对象!**

```
let tea={taste:"Good", fn:function(){return 1;}}; // 字面量(P44)
tea.taste; tea["taste"];  // 两种访问等价(P45) 方括号可用变量/保留字
tea.price=5; delete tea.price; for(let k in tea){} // 增删遍历(P46)
// globalThis/window(P47); this→取决于调用方式(P48)
```

this上下文(P48)	指向	this上下文	指向
方法中	所属对象	事件中	触发元素
单独/函数中	全局对象	严格模式	undefined

Constructor构造函数(P49)

```
function Drink(p){this.price=p;
  this.get=function(){return this.price*1.125;}} // ⚠每次new创新函数→浪费内存!
let d=new Drink(10); // new→空对象→调构造→this=新对象→返回
```

12. Prototype 原型与原型链 (P50–54) ★★

P50: JS不用class→**原型式prototype-based**(无类/面向原型/基于实例)

```
// 最佳: 方法放prototype上→所有实例共享!(P51)
function Drink(p){this.price=p;}        // 数据在实例
Drink.prototype.get=function(){         // 方法在prototype
  return this.price*1.125;};
let a=new Drink(5),b=new Drink(6);
a.get===b.get // true! 共享同一函数
```

原型链(P52) ★★ + 运行时扩展(P53)

```
查找: 实例自身 → Constructor.prototype → Object.prototype → null
a.hasOwnProperty("price")→true(自身)  a.hasOwnProperty("get")→false(prototype上)
// 运行时扩展(P53): 修改prototype→所有现有+未来实例立即可用
String.prototype.reversed=function(){...}; "live on".reversed()→"no evil"
```

new三步(P54): ①创对象proto=X.prototype ②调构造this=新对象 ③返回对象

Q: 实例属性和prototype同名? A: 实例**遮蔽shadow**prototype; delete实例属性后→沿链找到prototype的

13. 变量作用域 Scoping (P55–61) ★★

三种作用域(P56): Global=文档 | Function=函数内 | Block={}块内; 内层可见外层

声明	评级(P57)	作用域	可变	加window(P61)
`const`	Pure good	**Block**	✗	✗
`let`	Good	**Block**	✓	✗
`var`	Evil	**Function**	✓	✓
未声明	Pure Evil	**Global!**	✓	✓

P58-60同一结构对比 **★★必考**

```
function fun(c){if(c==0){
  const/let x=0 // P58: if内✓ if外✗ 函数外✗ ←Block 好!
  var x=0       // P59: if内✓ if外✓ 函数外✗ ←Function 泄漏!
  x=0           // P60: if内✓ if外✓ 函数外✓ ←Global 到处泄漏!
}}
```

P59: var**总是可替代**：不想块外用→改let; 想块外用→if前声明let
P61: 函数外声明(任何方式)→函数内可访问; 但**只有var加到window**

14. Closures 闭包 (P62–64) ★★

P62: 函数内可声明函数; 内部函数访问外层变量; **闭包=函数的局部变量在函数返回后仍存活**

```
// 课件经典: makeAdder (P63)
function makeAdder(a){        // a被闭包捕获
  return function(b){return a+b;}; // 返回的函数"记住"a
}
let add5=makeAdder(5);  // scope:{a:5}
let add20=makeAdder(20);// scope:{a:20} 独立!
add5(6)→11  add20(7)→27 // 互不干扰!
```

- 每次调用→新**scope object**保存局部变量(P63); 返回函数持有引用→**不被GC**(P64)
- vs全局(P64): ①每次调用创建**新**scope ②scope**不可**从代码直接访问

Q: for(var i=0;i<3;i++){setTimeout(()=>console.log(i),100);} 输出?
A: **3,3,3**。var函数级→一个i→循环结束i=3→都读3。改var为**let→0,1,2**(每次独立)

★ 易混淆概念辨析 Confusion Busters

混淆点	关键区别
null vs undefined	null=故意空(typeof→"object"!); undefined=未赋值; `null==undefined→T; ===→F`
== vs ===	==转类型(1==true→T); ===不转(1===true→F); **默认===**
slice vs splice	slice截取**不改原**返回新数组; splice插入/删除**改原**数组
push vs concat	push**改原**末尾加; concat**不改原**返回新数组
值传递 vs 共享传递	原始→副本不影响; 对象→改属性✓影响,重赋值✗不影响
构造内 vs prototype	构造内→每次new创新函数(浪费); prototype→所有实例**共享**(推荐)
fn vs fn()	`fn`=函数对象本身(可赋值); `fn()`=调用(得返回值)(P40)
for...in 对象vs数组	遍历**key**(属性名/索引字符串"0"不是值); 取值需`obj[key]`

★ 典型考题 Exam-Style Questions

Q1: 类型转换连环(P23-26)

```
1+"2"+3→"123"     // 1+"2"="12"→"12"+3="123"
1+2+"3"→"33"      // 1+2=3→3+"3"="33"
"5"-3+"1"→"21"    // "5"-3=2→2+"1"="21"
true+false+"1"→"11" // 1+0=1→1+"1"="11"
```

Q2: Call by Sharing追踪(P37)

```
function f(arr,n){arr.push(99);arr=[0];n=99;}
let a=[1,2],n=10; f(a,n);
// a=[1,2,99] ←push✓赋值✗  n=10 ←原始不变
```

Q3: Prototype遮蔽(P52)

```
function Cat(n){this.name=n;}
Cat.prototype.speak=function(){return "Meow";};
let c=new Cat("M"); c.speak=function(){return "Woof!";}
c.speak()→"Woof!"(自身遮蔽)  delete c.speak; c.speak()→"Meow"(回到proto)
```

Q4: 作用域综合(P58-61)

```
function test(){if(true){var a=1;let b=2;}
  console.log(a);  // 1 (var泄漏!)
  console.log(b);  // ReferenceError (let出块死!)
}
var x="v"; let y="l";
window.x→"v"(var加window)  window.y→undefined(let不加)
```

Q5: 闭包私有变量(P63-64)

```
function makeCounter(){
  let count=0; // 私有!外部不可访问
  return {inc:function(){return ++count;}, get:function(){return count;}};
}
let c1=makeCounter(),c2=makeCounter(); // 独立scope!
c1.inc();c1.inc(); // count=2
c2.inc();          // count=1 ←不受c1影响
```

★ 原型链图 + 易错速记

```
d(实例)          Drink.prototype      Object.prototype
┌──────────┐    ┌──────────────┐    ┌──────────────┐
│price: 10 │─proto→│getPrice: fn │─proto→│toString:fn │→null
└──────────┘    └──────────────┘    └──────────────┘

d.price→自身✓  d.getPrice()→proto✓  d.toString()→Object.proto✓  d.foo→undefined
```

```
// ASI(P13): return后换行→return;→undefined
// sort(P33): 默认字符串! [10,9,80].sort()→[10,80,9]  需(a,b)=>a-b
// delete(P33): arr中→undefined,length不变! delete a[1]→a=[1,undef,3]
// Array(P31): Array(3)→空len=3; Array(3,4)→[3,4]  ⚠一个数字vs多参数不同!
```